Module 1: Introduction to Mobile Computing

Weeks 2–4 · Second Semester, AY 2026–2027 · Seven lessons of 30 minutes each

Module overview

This module introduces mobile computing and mobile applications, and shows where front-end work fits in building them. It has seven lessons. Each is planned as a 30-minute class discussion with a timed outline, key points, a short activity and review questions. A seatwork and two quizzes follow, matching the assessments listed in the syllabus. The module prepares students for Module 2 (UI/UX design), Module 3 (Figma) and Module 4 (front-end frameworks with Ionic).

Item	Detail
Course	IT 322 – Track Elective 3 (Mobile App Development (Prog))
Schedule	Weeks 2–4, face-to-face, Second Semester, AY 2026–2027
Discussion time	7 lessons × 30 minutes = 3 hours 30 minutes, plus time for the seatwork and the two quizzes
Prerequisites	Basic computer skills and everyday smartphone use. No programming is needed.
Materials	Projector, laptop, board and paper. Slides for Lesson 1: Mobile_Computing.pptx
Program ILOs	IT01, IT02, IT05, IT07 (from the syllabus)
Assessments	Quiz 3 (20 items), Seatwork (15 items), Quiz 4 (20 items)
Running example	A campus canteen pre-order app, used in Lessons 5 to 7

Learning outcomes

At the end of the module, students are expected to:

1. Define what mobile computing is. *(Lesson 1)*
2. Enumerate the mobile applications. *(Lessons 2–3)*
3. Understand native and hybrid applications. *(Lesson 4)*
4. Understand the mobile development lifecycle. *(Lesson 5)*
5. Enumerate the mobile front-end workflow. *(Lessons 6–7)*

These are the syllabus outcomes. Each lesson adds its own, more specific objectives.

Module at a glance

Lesson	Topic	Discussion	Suggested week	Assessment after
1	What is Mobile Computing?	30 min	Week 2	
2	Mobile Applications	30 min	Week 2	
3	Mobile App Development	30 min	Week 2	Quiz 3 (Lessons 1–3, 20 items)
4	Native and Hybrid Applications	30 min	Week 3	
5	Mobile Development Lifecycle	30 min	Week 3	Seatwork (Lessons 4–5, 15 items)
6	What is Front-End?	30 min	Week 4	
7	Mobile Application Front-End Workflow	30 min	Week 4	Quiz 4 (Lessons 5–7, 20 items)
	Total discussion time	**3 h 30 min**		

Weeks are a suggestion based on the syllabus, which gives Weeks 2–4 to the whole module. Adjust the pacing to your class schedule.

Syllabus alignment

Teaching and learning activity in the syllabus	Where it happens
Lecture/discussion about what mobile computing is	Lesson 1
Lecture/discussion about how mobile computing affects mobile devices	Lesson 1, “How mobile computing shapes mobile devices”
Demonstration of the mobile development lifecycle	Lesson 5, Canteen Pre-Order walk-through
Lecture/demonstration of the front-end workflow	Lessons 6 and 7

How each 30-minute discussion runs

- **Same rhythm every time.** A hook of about 3 minutes, about 20 minutes of core discussion (with a demonstration in Lessons 5 and 7), a short activity of 3–6 minutes and a wrap-up of 2–4 minutes. Lesson 7 ends with a wrap-up instead of an activity. The timed plan at the start of each lesson gives the exact split.
- **If time runs short,** shorten the activity before the core discussion. The *Check yourself* questions can become an exit ticket or homework.
- **No devices needed.** The course policy restricts electronics in class, so activities use the board, paper and the teacher’s projector.
- **Answer keys** are in the last section. Remove it before sharing the module with students.

Lesson 1: What is Mobile Computing?

30 minutes of discussion · Syllabus ILO 1 · Slides: Mobile_Computing.pptx (7 slides)

By the end of this lesson, students can:

- Define mobile computing and name its three components, with an example of each.
- Explain how mobile computing shapes mobile devices, and state its advantages and limitations.

30-minute plan

Minutes	Segment	What happens
0–3	Hook	Pair talk: "What could you not do if you had no phone for a day?" Collect three answers.
3–10	Definition and components	Define mobile computing, introduce its three components and trace one GCash payment through them (slides 2–3).
10–17	How it shapes mobile devices	Walk through the table on how mobile computing shapes devices, then the evolution of devices and networks (slide 4).
17–24	Advantages and limitations	Slides 5–6. Pairs list two advantages and two limitations that matter where you live (signal, mobile data cost, battery).
24–30	Activity and wrap-up	"Spot the component" (4 min), then the key takeaways (slide 7, 2 min).

What is mobile computing?

Mobile computing is the use of portable computing devices that connect to networks wirelessly, so people can send and receive data, voice and video from anywhere without a fixed physical link. Two features define it: the devices are *portable*, and they stay *connected* over a network.

The three components

Component	What it covers	Examples
Mobile communication	The infrastructure that keeps links reliable: wireless networks, protocols, data formats, bandwidth and access points. Signals travel as radio waves.	Cellular networks (4G, 5G), Wi-Fi, Bluetooth
Mobile hardware	The portable devices and their parts, including the radio hardware that sends and receives signals.	Smartphones, tablets, laptops, smartwatches, e-readers
Mobile software	The operating system (the "brain" of the device) and the apps that run on it.	Android, iOS, Messenger, GCash

Example: you pay a friend with GCash. You tap *Send* in the app (**software**) on your phone (**hardware**), and the request travels over the mobile network (**communication**) to the servers and back.

How mobile computing shapes mobile devices

People expect to compute on the move, so every device and every app works within the same few demands.

Demand	How it shapes the device	What it means for apps
Be portable	Small, light bodies with touchscreens instead of a keyboard and mouse	Layouts for small screens and touch input
Work away from a power outlet	Rechargeable batteries and power-saving features	Apps should not drain the battery
Stay connected wirelessly	Built-in radios: cellular, Wi-Fi, Bluetooth and GPS	Apps must cope with slow, weak or lost connections
Know the user's situation	Sensors such as GPS, accelerometer, camera, microphone and fingerprint reader	Apps can use location, motion, photos and biometric login
Run on limited hardware	A scaled-down operating system (Android, iOS) with app stores and permissions	Apps are installed one by one and must ask permission for sensitive data

How it evolved

Networks	1G analog voice → 2G digital voice and SMS → 2.5G first packet data → 3G mobile internet → 4G broadband and HD video → 5G faster speeds, low delay and many connected devices
Devices	Laptops → PDAs → smartphones → tablets → wearables. Android and iOS now dominate; Windows Mobile and BlackBerry OS were discontinued.
Milestones	1973: first handheld mobile phone call · 1994: IBM Simon, often called the first smartphone · 2007: first iPhone · 2008: App Store and first Android phone

Advantages and limitations

Advantages	Limitations
Flexibility: work, study and communicate anywhere there is a connection	**Connectivity:** needs Wi-Fi or a mobile network; many apps do little without one
Productivity: less travel, quick access to files, and field staff who can collect and send data on the spot	**Security and privacy:** public Wi-Fi, lost or stolen devices and constant connectivity put data at risk
Services and entertainment: digital payments, online classes, streaming on the go	**Power:** batteries run out and charging points are not always available
Business: video meetings, secure remote access, lower travel costs	**Size and dependence:** small screens, limited storage and processing power; we feel lost when the phone fails

Activity: Spot the component (4 minutes)

On a sheet of paper, write **C** (communication), **H** (hardware) or **S** (software) for each item. Compare with a seatmate, then review as a class.

1) 5G network **2)** Smartphone touchscreen **3)** Android **4)** Wi-Fi hotspot **5)** GCash app **6)** Smartwatch

Check yourself

- Name the three components of mobile computing and give one example of each.
- Give two ways mobile computing influences how smartphones are designed.
- Name two advantages and two limitations of mobile computing.

Key terms: mobile computing, mobile communication, mobile hardware, mobile software, connectivity, sensor.

Lesson 2: Mobile Applications

30 minutes of discussion · Syllabus ILO 2

By the end of this lesson, students can:

- Define a mobile application and tell it apart from a website and from desktop software.
- Enumerate common categories of mobile apps, with examples.
- Explain where apps come from and name the main app stores.

30-minute plan

Minutes	Segment	What happens
0–4	Hook	Students write five apps they used yesterday, from memory (no phones). Tally the most common ones on the board.
4–9	What is a mobile app?	Definition and characteristics; how an app differs from a website and from desktop software.
9–20	Categories and examples	Walk through the categories table. Students sort their five apps into categories and add examples.
20–25	Where apps come from	Pre-installed and downloaded apps, app stores, and how apps earn money.
25–30	Activity and wrap-up	“Name that category” quick-fire (3 min), then review (2 min).

What is a mobile application?

A **mobile application**, or **app**, is application software designed to run on a mobile device such as a smartphone or tablet (Techopedia, 2020). Apps are usually small and focused: each does one job, such as messaging, navigation or banking, and users decide which ones to install. Most apps:

- are designed for touch, small screens and quick, one-handed use;
- can use device features such as the camera, microphone, GPS, sensors, contacts and notifications, with the user’s permission;
- are installed from an app store, or come pre-installed on the phone;
- connect to online services for accounts, data and payments.

	Mobile app	Mobile website	Desktop software
Where it runs	Installed on the phone or tablet	In the browser; nothing to install	Installed on a computer
Device features	Camera, GPS, sensors, notifications (with permission)	Limited; depends on the browser	Depends on the computer; mouse and keyboard
Offline use	Often possible	Usually needs a connection	Often possible
Typical scope	Small and focused	A whole site	Large, many functions
Distribution	App stores	A web address (URL)	Download or installer

The lines blur: progressive web apps (Lesson 4) make websites behave more like apps.

Categories of mobile apps

Stores group apps by purpose. These ten categories cover most of what students use every day.

Category	What it does	Examples
Communication and social	Messaging, calls and social networking	Messenger, Viber, Facebook, TikTok
Games	Play, compete and relax	Mobile Legends: Bang Bang, Roblox, Call of Duty: Mobile
Entertainment and media	Video, music and streaming	YouTube, Netflix, Spotify
Productivity and business	Documents, meetings and teamwork	Google Docs, Microsoft Teams, Google Meet
Education	Classes, practice and learning	Google Classroom, Duolingo, Khan Academy
Finance and payments	Mobile wallets and banking	GCash, Maya
Shopping	Buying and selling online	Shopee, Lazada
Travel, maps and transport	Navigation and ride-hailing	Google Maps, Waze, Grab
Health and fitness	Activity, sleep and wellness tracking	Samsung Health, Strava
Utilities and tools	Small everyday tools	Calculator, Camera, Flashlight, Google Translate

Many apps fit more than one category (Grab covers rides, food and payments). Classify by main purpose.

Where apps come from

- **Pre-installed apps** ship with the phone, such as Phone, Camera, Messages and Settings.
- **Downloaded apps** come from an app store. The two main ones are Google Play (Android) and the Apple App Store (iOS); others include the Samsung Galaxy Store and Huawei AppGallery. Stores review apps before publishing them.
- **Sideloading** means installing an Android app from outside a store. It skips that review and carries a higher security risk.
- **How apps earn:** free with ads, paid, free with in-app purchases (freemium), or by subscription.

Activity: Name that category (3 minutes)

The teacher names an app and the class calls out its category: Duolingo, Waze, Spotify, Maya, Shopee, Mobile Legends, Google Classroom.

Check yourself

- In your own words, what is a mobile app?
- List four categories of mobile apps and give one example of each.
- Give two advantages an installed app can have over a website opened in a browser.

Key terms: mobile application (app), app store, pre-installed app, sideloading, in-app purchase, freemium.

Lesson 3: Mobile App Development

30 minutes of discussion · Supports syllabus ILOs 2–4

By the end of this lesson, students can:

- Define mobile app development and say what makes it different from desktop or web development.
- Name the two dominant platforms and their official tools.
- List the roles in an app team and the main challenges of building for mobile.

30-minute plan

Minutes	Segment	What happens
0–3	Hook	"You have an app idea. What do you need to get it onto someone's phone?" List the answers on the board.
3–8	What is mobile app development?	Definition, and what makes building for mobile different from building for desktop or web.
8–16	Platforms and tools	Android and iOS compared, cross-platform options, emulators and simulators.
16–22	Team roles and challenges	Who builds an app, and what makes mobile development hard.
22–30	Activity and wrap-up	"Pitch your app" in pairs (6 min), then link the pitches back to the board list (2 min).

What is mobile app development?

Mobile application development is the process of creating software that runs on a mobile device. Most apps use a network connection to work with remote resources, so the job includes building the installable app package (code and assets), creating back-end services such as data access through an API, and testing the app on the target devices (AWS, n.d.).

It differs from desktop and web development because of small screens and touch input, limited battery and memory, sensors and permissions, changing network quality, distribution through app stores, and many device models and OS versions.

Platforms and tools

Two platforms dominate the smartphone market: **Android** (Google) and **iOS** (Apple). Developing for each uses a different SDK and toolchain (AWS, n.d.). In the Philippines, StatCounter's October 2026 figures, which measure web traffic, show Android with about 86% of mobile OS share and iOS with about 14% (StatCounter, 2026).

	Android	iOS
Made by	Google; also used by many phone makers	Apple; iPhone only
Main languages	Kotlin (Google's preferred language), Java	Swift (Objective-C in older apps)
Official IDE	Android Studio (Windows, macOS, Linux)	Xcode (macOS only)
Virtual testing	Android Emulator	iOS Simulator
Main app store	Google Play (other stores exist)	Apple App Store

An **SDK** (software development kit) bundles the libraries, tools and documentation for one platform. An **IDE** (integrated development environment) is the editor and toolset where developers write, run and debug code. **Emulators and simulators** run a virtual phone on a computer, but testing on real devices is still needed.

Cross-platform tools let one team target both platforms from a single codebase: Ionic (used in Module 4), React Native, Flutter and .NET MAUI. Lesson 4 compares these approaches.

Who builds an app?

Role	What they do
Product owner or business analyst	Decides what the app must do and for whom
UI/UX designer	Plans how the app looks and feels (Modules 2 and 3)
Front-end (mobile) developer	Builds the screens users see and touch (Lessons 6–7, Module 4)
Back-end developer	Builds servers, databases and APIs
QA tester	Finds bugs and checks quality on many devices
Project manager	Keeps the schedule, budget and team on track

In small teams, such as student projects, one person often plays several roles.

Common challenges

- **Device fragmentation:** many brands, screen sizes and OS versions, especially on Android.
- **Limited resources:** battery, memory and storage.
- **Unreliable connectivity:** apps must handle slow or missing signal.
- **Security and privacy:** apps often handle personal data. In the Philippines, the Data Privacy Act of 2012 (Republic Act No. 10173) applies.
- **Store rules and updates:** stores review apps, and new OS versions can break existing ones.

Activity: Pitch your app (6 minutes)

In pairs, choose a campus app idea, such as canteen pre-ordering, class schedule reminders or a lost-and-found board. On paper, write:

1. Who will use it.
2. Three core features.
3. Which platform or platforms to target, and why.
4. Which team roles you would need.

Two pairs share their pitch with the class.

Check yourself

- Name two ways mobile development differs from desktop development.
- Which language and IDE would you use for a native Android app? For a native iOS app?
- Why do many teams choose a cross-platform approach?

Key terms: mobile app development, SDK, IDE, emulator, API, device fragmentation, cross-platform.

Lesson 4: Native and Hybrid Applications

30 minutes of discussion · Syllabus ILO 3

By the end of this lesson, students can:

- Define native and hybrid apps and compare them on performance, device access, cost and look and feel.
- Name cross-platform native frameworks and progressive web apps as other options.
- Choose an approach for a given scenario and justify the choice.

30-minute plan

Minutes	Segment	What happens
0–3	Hook	“If one codebase could run on both Android and iPhone, what might you give up?” Take two or three answers.
3–10	Native apps	Definition, tools, strengths and weaknesses.
10–17	Hybrid apps	How the web-in-a-container model works, its strengths and weaknesses, and the side-by-side table.
17–22	Beyond the two	Cross-platform native frameworks, progressive web apps and how to choose.
22–30	Activity and wrap-up	“Which approach?” with four scenarios (6 min), then wrap-up (2 min).

Native apps

A **native app** is written in the language and tools provided by the platform owner and runs directly on the device’s operating system (AWS, n.d.). Android apps use Kotlin or Java with the Android SDK; iOS apps use Swift or Objective-C with the iOS SDK. A native app works only on the platform it was built for, so supporting both platforms means two codebases (Techopedia, 2024).

- **Strengths:** best performance, full and immediate access to device features, and a look and feel that matches the platform.
- **Weaknesses:** higher cost and longer development time (two codebases and two skill sets), and separate updates for each platform.

Hybrid apps

A **hybrid app** is built with standard web technologies (HTML, CSS and JavaScript or TypeScript) and packaged inside a native container that shows the web content in an embedded browser view, called a *WebView*. The app is installed like any other and behaves like a native app (AWS, n.d.). A plugin layer, such as Capacitor or Apache Cordova, lets the web code reach the camera, GPS and other device features. Ionic, which this course uses in Module 4, is a hybrid framework.

- **Strengths:** one codebase for Android and iOS, faster and cheaper development, and web developers can reuse the skills they already have.
- **Weaknesses:** performance can lag in graphics-heavy or animation-heavy apps, new device features depend on plugin support, and the interface needs extra care to feel at home on each platform.

	Native	Hybrid
Built with	The platform's own language and SDK (Kotlin or Java, Swift)	HTML, CSS and JavaScript or TypeScript inside a native container
Codebase	One for each platform	One for all platforms
Performance	Best	Good for most apps; weaker for heavy graphics
Device features	Full and immediate	Through plugins
Look and feel	Matches the platform	Close, but needs tuning
Cost and time	Higher	Lower
Example tools	Android Studio, Xcode	Ionic with Capacitor, Apache Cordova

Two more approaches, and how to choose

AWS lists four ways to build mobile apps: native, cross-platform native, hybrid and progressive web apps (AWS, n.d.).

- **Cross-platform native:** one codebase, but the result is a native app. Frameworks such as React Native, Flutter and .NET MAUI render native interface elements or compile to native code. Some people call these "hybrid" too; in this course, *hybrid* means web code inside a native container.
- **Progressive web app (PWA):** a web app that runs in the browser, can be added to the home screen and can work offline, but has more limited access to device features.

How to choose: weigh the user experience you want, the device features and computing power the app needs, your budget and deadline, and the resources you have to maintain the app.

Activity: Which approach? (6 minutes)

In pairs, write **Native** or **Hybrid** for each scenario and give one reason.

A. A 3D racing game with heavy graphics for Android and iPhone.

B. An events-and-announcements app for a student organization, built by two developers in six weeks for Android and iPhone.

C. An app that must use brand-new iPhone camera features on the day they are released.

D. A company with a mobile-friendly website that wants an app-store presence quickly.

Check yourself

- In one sentence, what makes an app hybrid?
- Give two advantages of native apps and two of hybrid apps.
- Why might a class with limited time and mixed devices choose a hybrid framework such as Ionic?

Key terms: native app, hybrid app, WebView, plugin, cross-platform, progressive web app (PWA), codebase.

Lesson 5: Mobile Development Lifecycle

30 minutes of discussion · Syllabus ILO 4 · Includes the lifecycle demonstration

By the end of this lesson, students can:

- Name the six stages of the lifecycle in order.
- Describe what happens, and what is produced, at each stage.
- Explain why mobile development is iterative, using Agile, sprints and the MVP.

30-minute plan

Minutes	Segment	What happens
0–3	Hook	"Think of an app that updated this week. Why do apps keep changing after release?"
3–7	Overview	The six stages as a loop.
7–20	Demonstration	Trace the Canteen Pre-Order app through the six stages, naming the key question and the output at each one.
20–25	Iterative development	Waterfall versus Agile, sprints and the minimum viable product (MVP).
25–30	Activity and wrap-up	"Put the tasks in order" (3 min), then review (2 min).

The six stages

Discovery and planning → **Design** → **Development** → **Testing** → **Deployment** → **Maintenance**

What teams learn after release feeds the next round of planning, so the stages form a loop. Sources name and group the stages a little differently; this module uses these six.

Stage	Key question	What happens	Typical output
1. Discovery and planning	What problem are we solving, and for whom?	Research users and competitors, set goals, list requirements, choose platforms and an approach (native or hybrid), estimate budget and time	Requirements list, project plan
2. Design	How should it look and work?	Map user flows, sketch wireframes, design the screens, build a clickable prototype	Wireframes, mock-ups, prototype
3. Development	Can we build it?	Code the front end (screens) and the back end (server, database, APIs) in short cycles, using version control	Working builds of the app
4. Testing	Does it work for everyone, everywhere?	Test features, usability, performance, security and compatibility on different devices, screen sizes and OS versions; fix bugs; run a beta test	Test results, release candidate
5. Deployment	How do users get it?	Prepare the store listing (name, description, icon, screenshots, privacy policy), build the release, submit for store review and publish	App live in Google Play or the App Store
6. Maintenance	How do we keep it working and improve it?	Monitor crashes, reviews and usage; fix bugs; update for new OS versions; add features; retire the app when its time is over	Updates (v1.1, v1.2, …)

Demonstration: the Canteen Pre-Order app

A team builds an app that lets students order canteen food ahead and pick it up without queuing.

Stage	What the team does
1. Planning	Interviews students and canteen staff. The first version (MVP) lets students browse the menu, order ahead and get a "ready" notification. The team knows web technologies, so it chooses a hybrid approach.
2. Design	Sketches screens on paper, draws wireframes for Menu, Cart and Order Status, and builds a clickable prototype that three students try.
3. Development	Builds the screens (front end) and the menu-and-orders database with its API (back end) in two-week cycles.
4. Testing	Tests on a low-end Android phone, a recent iPhone, slow data and no signal; fixes bugs; invites 20 students to a beta test.
5. Deployment	Writes the store description, adds screenshots and a privacy policy, submits to Google Play and publishes.
6. Maintenance	Reads crash reports and reviews, releases version 1.1 to fix a crash, adds GCash payment in 1.2 and updates the app for a new Android version.

Optional live demo: open any popular app's Google Play page in a browser and point out lifecycle evidence: *Updated on* and *What's new* (maintenance), ratings and reviews (user feedback) and the *Data safety* section (a release requirement).

Iterative development

The lifecycle is a loop, not a straight line. What teams learn after release becomes the input for the next round of planning.

- **Waterfall:** each stage is finished before the next one starts. It is easy to plan but slow to react to change.
- **Agile:** the team works in short cycles called sprints, often one to four weeks. Each sprint produces a working piece of the app and is adjusted with feedback. Most mobile teams work this way.
- **MVP (minimum viable product):** the smallest version of the app that is useful enough to release and learn from.

Activity: Put the tasks in order (3 minutes)

Write 1 to 6 beside the tasks to show the order a team would do them in, then name the lifecycle stage of each.

- Submit the app for Google Play review
- Interview students about what they need
- Fix a crash and release version 1.1
- Code the login screen
- Draw wireframes of the main screens
- Test the app on a slow network

Check yourself

- List the six stages of the lifecycle in order.
- Why is the lifecycle drawn as a loop?
- In which stage would you find (a) wireframes, (b) a store listing, (c) a crash report?

Key terms: lifecycle, requirements, prototype, testing, deployment, maintenance, waterfall, Agile, sprint, MVP.

Lesson 6: What is Front-End?

30 minutes of discussion · Syllabus ILO 5

By the end of this lesson, students can:

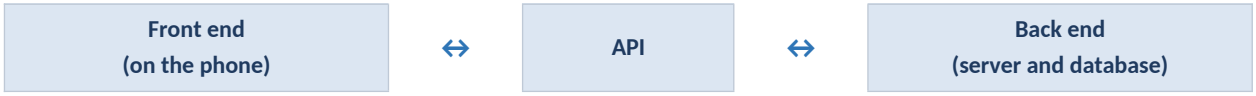
- Define the front end and the back end of a mobile app, and how they communicate.
- Describe what a front-end developer is responsible for.
- Relate front-end work to the tools used in this course, and classify tasks as front end or back end.

30-minute plan

Minutes	Segment	What happens
0–3	Hook	Restaurant analogy: the dining area and the kitchen. “What is the front end of a food-ordering app?”
3–10	Front end and back end	Definitions, the device–API–server diagram, and the analogy mapped to an app.
10–18	What the front end does	Walk through the seven responsibilities. For each one, students suggest an example from the Canteen Pre-Order app.
18–24	Front-end technologies	Tools for each approach, where Ionic fits, and the designer and developer roles.
24–30	Activity and wrap-up	“Front or back?” (4 min), then wrap-up (2 min).

Front end and back end

The **front end** is the part of an app that users see and touch. It runs on the user’s device. The **back end** runs on servers: it stores data, applies business rules, checks identities and handles payments. The two talk through an **API** (application programming interface), a set of requests and responses the app uses to ask the server for data or to send data to it.



In a restaurant	In a mobile app
The dining area, menu and table: what customers see and use	**Front end:** screens, buttons, menus
The kitchen, where the food is prepared	**Back end:** server, database, business rules
The waiter, who carries orders to the kitchen and food back	**API:** carries requests and responses between them

What the user sees (front end)	What happens behind the scenes (back end)
Menu list with photos and prices	A database stores the items and prices
“Add to cart” button and cart screen	The server saves the order and works out the total
Login form with an “Enter a valid email” hint	The server checks the password and starts the session
“Place order” button with a progress spinner	The server confirms payment and sends the order to the kitchen
“Your order is ready” alert on the screen	The server triggers the notification when the kitchen marks the order done

What the front end does

1. **Layout and visual design:** screens, colors, fonts, icons and spacing.
2. **Navigation:** moving between screens with tabs, menus, buttons and gestures.
3. **User input:** taps, swipes, typing and form validation.
4. **Showing data:** lists and images from the back end, including loading, empty and error states.
5. **Responsiveness:** adapting to different screen sizes and orientations.
6. **Accessibility and performance:** readable text, large enough touch targets and fast screens.
7. **Client-side logic:** keeping state on the device and caching data for offline use.

Validation happens on both sides. The front end gives quick feedback ("password too short"). The back end repeats the checks for security, because the front end runs on a device the user controls.

Front-end technologies and roles

Approach	Typical front-end tools
Native Android	Kotlin (or Java), Jetpack Compose or XML layouts, Android Studio
Native iOS	Swift, SwiftUI or UIKit, Xcode
Hybrid (this course)	HTML, CSS and TypeScript with a framework such as Ionic with Angular, written in a code editor such as VS Code
Cross-platform native	React Native (JavaScript or TypeScript) or Flutter (Dart)
Design	Figma (Module 3)

A **front-end framework** is a ready-made toolkit of interface components and structure, so developers do not build every button, list and menu from scratch. Module 4 covers Ionic.

The **UI/UX designer** decides how the app should look and behave; the **front-end developer** turns that design into working screens. In small teams, one person often does both.

Activity: Front or back? (4 minutes)

Write **F** (front end) or **B** (back end) for each task.

1. A button that adds an item to the cart
2. The database that stores user accounts
3. A spinner shown while the menu loads
4. Checking that the password is correct
5. Choosing the app's colors and fonts
6. Swiping left to delete an item from the cart
7. Confirming the final payment with the payment provider
8. Sending a "Your order is ready" notification from the server

Check yourself

- In your own words, what is the front end of a mobile app?
- Name three responsibilities of a front-end developer.
- Why must the back end repeat the checks that the front end makes?

Key terms: front end, back end, API, UI, UX, client side, server side, front-end framework.

Lesson 7: Mobile Application Front-End Workflow

30 minutes of discussion · Syllabus ILO 5 · Includes the front-end workflow demonstration

By the end of this lesson, students can:

- Enumerate the seven steps of the front-end workflow in order.
- State what is produced at each step and which course module covers it.
- Explain why the steps are repeated rather than done once.

30-minute plan

Minutes	Segment	What happens
0–3	Hook	"Would you start building a house by laying bricks?" Why does front-end work also need a plan?
3–7	The big picture	The seven steps, and where they sit inside the lifecycle (Lesson 5).
7–21	Demonstration	Walk the Canteen Pre-Order app through the seven steps, about two minutes each, showing a sample output for each.
21–26	Tools, hand-off and iteration	Tools for each step, how designers hand work to developers, and why steps repeat.
26–30	Wrap-up	Roadmap to Modules 2–4 and the exit question.

The seven steps

The front-end workflow zooms in on the user-facing work inside the lifecycle (Lesson 5). It starts with the planning questions about users, then moves through Design, Development and Testing.



#	Step	What happens	Output	Module
1	Understand users and requirements	Find out who will use the app, which tasks they must finish and on which devices	User profiles, feature list	Module 2
2	Map the user flow and screens	List the screens and how users move between them	Screen list, user-flow diagram	Module 2
3	Wireframe	Sketch the layout of each screen without color or graphics; refine from low to mid to high fidelity	Wireframes	Module 2
4	Design the user interface	Choose colors, typography, icons and spacing; design each screen and reusable component	UI mock-ups, style guide	Modules 2–3
5	Prototype and get feedback	Link the screens into a clickable prototype, let real users try it, then revise	Clickable prototype, feedback notes	Module 3
6	Build the front end	Set up the project, turn the design into screens and components, add navigation and styling, connect to back-end data	Working front end	Module 4
7	Test and refine	Run on emulators and real devices, screen sizes and OS versions; check performance and accessibility; fix bugs and polish	Stable front end, ready for release	Module 4

Demonstration: the Canteen Pre-Order app

Step	Sample output to show
1	Users: students with short breaks and canteen staff. Tasks: order ahead, pay, pick up.
2	Flow: Login → Menu → Item details → Cart → Order status.
3	A paper sketch of the Menu screen: search bar on top, food cards in a list, tab bar at the bottom.
4	A green-and-orange theme, 16-point body text, rounded buttons and the same card design on every list.
5	Testers tap *Add to cart* but cannot find *Checkout*, so the button moves to the bottom of the cart screen.
6	A project with Menu, Cart and Order Status pages, with the menu loaded from the API.
7	Tests on an Android emulator and a low-end phone; text that is cut off on small screens gets fixed.

Preparing the demonstration

Prepare one sample output per step before class: a user-flow sketch, a paper wireframe, a style tile, a mock-up screen, a short prototype clip and a screenshot of a running app. Photos of paper sketches work well. The real tools come in Modules 2–4.

Tools, hand-off and iteration

- **Tools:** paper and whiteboard (steps 1–3), Figma (steps 4–5), a code editor with Ionic (step 6), and emulators plus real devices (step 7).
- **Hand-off:** the designer shares the final screens, assets (icons and images) and specifications (colors, font sizes, spacing) so the developer can match the design.
- **Iteration:** problems found in the prototype (step 5) send you back to steps 3 and 4; bugs found in testing (step 7) send you back to step 6. Fixing a problem on a wireframe costs far less than fixing it after the code is written.

Wrap-up (4 minutes)

Roadmap. Module 2 covers steps 1 to 4, Module 3 covers steps 4 and 5, and Module 4 covers steps 6 and 7.

Exit question. Which step do teams skip most often, and what goes wrong when they do?

Check yourself

- Enumerate the seven steps of the front-end workflow in order.
- What is the difference between a wireframe and a prototype?
- Why should a prototype be tested before any code is written?

Key terms: user flow, wireframe, mock-up, prototype, hand-off, style guide, emulator.

Seatwork: Native, Hybrid and the Lifecycle

Covers Lessons 4–5 · 15 items · Suggested time: 20 minutes · 1 point per item

Name: _____ Section: _____ Date: _____ Score: _____ / 15

Part A. Native or hybrid? (items 1–6)

Write N on the blank if the statement describes a native app, or H if it describes a hybrid app.

_____ 1. Built with HTML, CSS and JavaScript and packaged inside a native container.

_____ 2. Built with Kotlin and Android Studio.

_____ 3. Needs a separate codebase for each platform.

_____ 4. Uses plugins such as Capacitor or Cordova to reach the camera and other device features.

_____ 5. Lets web developers reuse their HTML, CSS and JavaScript skills.

_____ 6. Gives the best performance and immediate access to the newest device features.

Part B. Lifecycle stages (items 7–12)

Write the letter of the lifecycle stage that best fits each activity.

Stages

A. Discovery and planning **B.** Design **C.** Development **D.** Testing **E.** Deployment **F.** Maintenance

_____ 7. Preparing screenshots and a privacy policy and submitting the app to Google Play.

_____ 8. Interviewing students to list what the app must do.

_____ 9. Running the app on a slow network and on a low-end phone to look for crashes.

_____ 10. Releasing version 1.1 to fix a crash that users reported.

_____ 11. Drawing wireframes of the main screens.

_____ 12. Coding the login screen and connecting it to the server.

Part C. Choose the approach (items 13–15)

Write Native or Hybrid for each scenario, then give one reason. Each item is worth 1 point: ½ point for the choice and ½ point for the reason.

13. An augmented-reality measuring app for iPhone that relies heavily on the camera and motion sensors.

    Approach: _____ Reason: _____

14. A school's class-schedule and announcements app for Android and iPhone, built in one semester by two students who know HTML, CSS and TypeScript.

    Approach: _____ Reason: _____

15. A start-up with a small budget must launch a simple appointment-booking app on both app stores within a month.

    Approach: _____ Reason: _____

Quiz 3: Mobile Computing, Mobile Applications and Mobile App Development

Covers Lessons 1–3 · 20 items · Suggested time: 20 minutes · 1 point per item

Name: _____ Section: _____ Date: _____ Score: _____ / 20

Test I. Multiple choice (items 1–10)

Write the letter of the best answer on the blank before each number.

_____ 1. Which statement best describes mobile computing?
- a) Computing on a desktop computer connected to the network by a cable
- b) Storing all files on one personal computer
- c) Computing on portable devices that connect wirelessly, without a fixed physical link
- d) Making phone calls only

_____ 2. Which is NOT one of the three components of mobile computing?
- a) Mobile advertising
- b) Mobile communication
- c) Mobile hardware
- d) Mobile software

_____ 3. A smartphone's operating system, such as Android or iOS, belongs to which component?
- a) Mobile communication
- b) Mobile hardware
- c) Mobile network
- d) Mobile software

_____ 4. A student cannot submit an online form because there is no Wi-Fi or mobile signal. Which limitation of mobile computing does this show?
- a) Power
- b) Connectivity
- c) Security
- d) Screen size

_____ 5. Which sensor lets an app detect that the phone is being tilted?
- a) GPS
- b) Microphone
- c) Accelerometer
- d) SIM card

_____ 6. Which design feature of mobile devices comes directly from the need to be portable?
- a) Wired network ports
- b) Large fixed monitors
- c) An external keyboard that is required to use the device
- d) Small, light bodies with touchscreens

_____ 7. According to Techopedia, a mobile application is application software designed to run on:
- a) A mobile device, such as a smartphone or tablet
- b) A mainframe computer
- c) A web server only
- d) A printer

_____ 8. GCash and Maya belong to which category of mobile apps?

a) Games
b) Finance and payments
c) Health and fitness
d) Utilities and tools

_____ 9. Which is the official IDE for building native Android apps?

a) Xcode
b) Figma
c) Android Studio
d) Microsoft Word

_____ 10. Which statement about mobile app development is correct?

a) Developers can ignore battery life and network quality.
b) Only one team role is needed for every app.
c) Apps never need testing on real devices.
d) Apps usually have to run well on many device models, screen sizes and OS versions.

Test II. True or false (items 11–14)

Write TRUE if the statement is correct. Write FALSE if it is not.

_____ 11. Mobile computing requires a fixed physical link between the device and the network.

_____ 12. A mobile app and a mobile website are exactly the same thing.

_____ 13. Xcode, the official tool for building native iOS apps, runs only on macOS.

_____ 14. A typical mobile app works with back-end services over a network connection, for example to get or save data through an API.

Test III. Enumeration (items 15–20)

Write one answer on each line.

15–17. Name the three components of mobile computing.

15. _____
16. _____
17. _____

18–20. Name any three categories of mobile apps.

18. _____
19. _____
20. _____

Quiz 4: Lifecycle, Front End and Front-End Workflow

Covers Lessons 5–7 · 20 items · Suggested time: 20 minutes · 1 point per item

Name: _____ Section: _____ Date: _____ Score: _____ / 20

Test I. Multiple choice (items 1–9)

Write the letter of the best answer on the blank before each number.

_____ 1. Which is the correct order of the stages of the mobile development lifecycle?
- a) Design, planning, testing, development, deployment, maintenance
- b) Planning, development, design, deployment, testing, maintenance
- c) Development, planning, design, maintenance, testing, deployment
- d) Planning, design, development, testing, deployment, maintenance

_____ 2. In which stage are wireframes and prototypes created?
- a) Design
- b) Testing
- c) Deployment
- d) Maintenance

_____ 3. Preparing a store listing and submitting the app for review belongs to which stage?
- a) Planning
- b) Development
- c) Deployment
- d) Maintenance

_____ 4. Why is the mobile development lifecycle often drawn as a loop?
- a) Because apps are tested only after they are retired
- b) Because feedback after release leads to new planning and updates
- c) Because every stage must be done twice
- d) Because designers and developers take turns

_____ 5. Which part of a mobile app is the front end?
- a) The database that stores user accounts
- b) The server that confirms payments
- c) The screens, buttons and menus that users see and touch
- d) The code that sends notifications from the server

_____ 6. In the restaurant analogy, the API is like:
- a) The dining area
- b) The kitchen
- c) The tables and chairs
- d) The waiter who carries orders and food between the two

_____ 7. Which task is mainly a front-end responsibility?
- a) Showing a loading spinner while data downloads
- b) Designing the database tables
- c) Confirming payment with the payment provider
- d) Storing passwords securely on the server

_____ 8. A front-end framework is best described as:

a) A tool that stores data on a server
b) A ready-made toolkit of interface components and structure for building screens
c) A type of database
d) A store where apps are published

_____ 9. In the front-end workflow, which step comes right after creating wireframes?

a) Build the front end
b) Test and refine
c) Design the user interface
d) Understand users and requirements

Test II. True or false (items 10–13)

Write TRUE if the statement is correct. Write FALSE if it is not.

_____ 10. A minimum viable product (MVP) is the smallest version of an app that is useful enough to release and learn from.

_____ 11. The back end is the part of an app that runs on the user's phone and shows the screens.

_____ 12. A prototype is tested with real users before the code is written, so problems can be fixed early.

_____ 13. Because the front end already checks the data, the back end does not need to check it again.

Test III. Enumeration (items 14–20)

Enumerate, in order, the seven steps of the mobile front-end workflow. Write one step on each line. Equivalent wording is accepted.

14. _____
15. _____
16. _____
17. _____
18. _____
19. _____
20. _____

References and further reading

Cited in this module

Amazon Web Services. (n.d.). *Mobile application development*. Amazon Web Services, Inc.

Amazon Web Services. (n.d.). *What's the difference between web apps, native apps, and hybrid apps?* Amazon Web Services, Inc.

Nambiar, K. (2021, September 1). *Introduction to mobile computing*. Analytics Steps.

Republic Act No. 10173. (2012). *Data Privacy Act of 2012*. Republic of the Philippines.

StatCounter. (2026, October). *Mobile operating system market share in the Philippines*. StatCounter Global Stats.

Techopedia. (2020, August 7). *What is a mobile application? Definition from Techopedia*. Techopedia.com.

Techopedia. (2024, July 19). *What is a native mobile app?* Techopedia.com.

Online resources

Introduction to mobile computing (Analytics Steps): https://www.analyticssteps.com/blogs/introduction-mobile-computing

Mobile application development (AWS): https://aws.amazon.com/mobile/mobile-application-development/

Web apps, native apps and hybrid apps (AWS): https://aws.amazon.com/compare/the-difference-between-web-apps-native-apps-and-hybrid-apps/

What is a native mobile app? (Techopedia): https://www.techopedia.com/definition/27568/native-mobile-app

Mobile operating system market share, Philippines (StatCounter): https://gs.statcounter.com/os-market-share/mobile/philippines

Data Privacy Act of 2012 (National Privacy Commission): https://www.privacy.gov.ph/data-privacy-act/

Further reading for later modules

Ionic Framework documentation (Module 4): https://ionicframework.com/docs

Android developer documentation: https://developer.android.com/

Apple developer documentation: https://developer.apple.com/documentation/

Teacher's answer keys

> **For the teacher only**
> This section starts on its own page so you can remove it before sharing the module with students.

Seatwork key (15 points)

Item	1	2	3	4	5	6	7	8	9	10	11	12
Answer	H	N	N	H	H	N	E	A	D	F	B	C

Items 1–6: N = native, H = hybrid. Items 7–12: A = Discovery and planning, B = Design, C = Development, D = Testing, E = Deployment, F = Maintenance.

Item	Choice	Accept a reason such as
13	Native	Heavy use of the camera and motion sensors needs full device access and top performance.
14	Hybrid	The team knows web technologies, one codebase covers both platforms, and the deadline is short.
15	Hybrid	One codebase lowers cost and time. A cross-platform native framework is also a good answer if justified.

Quiz 3 key (20 points)

Item	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Answer	c	a	d	b	c	d	a	b	c	d	F	F	T	T

Items 11–14: T = TRUE, F = FALSE. **Items 15–17:** mobile communication, mobile hardware, mobile software (any order). **Items 18–20:** any three categories, such as communication and social, games, entertainment and media, productivity and business, education, finance and payments, shopping, travel, maps and transport, health and fitness, or utilities and tools. Accept other reasonable categories, such as news or photography.

Quiz 4 key (20 points)

Item	1	2	3	4	5	6	7	8	9	10	11	12	13
Answer	d	a	c	b	c	d	a	b	c	T	F	T	F

Items 10–13: T = TRUE, F = FALSE. **Items 14–20,** in order: (14) understand users and requirements; (15) map the user flow and screens; (16) wireframe; (17) design the user interface; (18) prototype and get feedback; (19) build the front end; (20) test and refine. Give 1 point for each step that is named correctly and appears in the right order relative to the other steps.

Lesson activity answers

Lesson	Activity	Answers
1	Spot the component	1 C · 2 H · 3 S · 4 C · 5 S · 6 H
2	Name that category	Duolingo: Education · Waze: Travel, maps and transport · Spotify: Entertainment and media · Maya: Finance and payments · Shopee: Shopping · Mobile Legends: Games · Google Classroom: Education
3	Pitch your app	No fixed answer. Look for a clear user, three specific features, a platform choice with a reason, and realistic team roles.

Lesson	Activity	Answers
4	Which approach?	A Native (full graphics performance). B Hybrid (one codebase, web skills, short deadline). C Native, iOS (immediate access to new platform features). D Hybrid (reuse the existing web code).
5	Put the tasks in order	1 Interview students (Planning) · 2 Draw wireframes (Design) · 3 Code the login screen (Development) · 4 Test on a slow network (Testing) · 5 Submit for Google Play review (Deployment) · 6 Fix a crash and release v1.1 (Maintenance)
6	Front or back?	1 F · 2 B · 3 F · 4 B · 5 F · 6 F · 7 B · 8 B
7	Exit question	Open-ended. A strong answer names a step (often 1, 2 or 5) and a consequence, such as building features nobody needs or redoing finished code.

Model answers for the Check yourself questions

Lesson	Model answers
1	(1) Communication: cellular network, Wi-Fi. Hardware: smartphone, tablet. Software: Android, iOS or an app. (2) Any two: small size and touchscreens for portability; rechargeable batteries and power saving; built-in radios for connectivity; sensors for context. (3) Advantages: flexibility, saved time, productivity, field work, entertainment, business. Limitations: connectivity, security, power, size, dependence.
2	(1) Application software designed to run on a mobile device, usually small and focused on one job. (2) Any four categories from the table, each with a matching example. (3) Any two: faster and more responsive, access to device features, offline use, push notifications, discoverability in app stores.
3	(1) Any two: small screens and touch input, battery and memory limits, sensors and permissions, changing connectivity, app stores, many device models and OS versions. (2) Android: Kotlin (or Java) with Android Studio. iOS: Swift with Xcode. (3) One codebase for both platforms means lower cost and faster delivery.
4	(1) It is built with web technologies and packaged in a native container. (2) Native: best performance, full device access, platform look and feel. Hybrid: one codebase, lower cost, faster delivery, reuse of web skills. (3) Students can reuse web skills, build one codebase for both platforms and finish within the semester.
5	(1) Discovery and planning, design, development, testing, deployment, maintenance. (2) Feedback and problems found after release lead to new planning and updates. (3) (a) Design, (b) Deployment, (c) Maintenance.
6	(1) The part of the app users see and touch; it runs on their device. (2) Any three: layout and visual design, navigation, user input, showing data, responsiveness, accessibility and performance, client-side logic. (3) The front end runs on a device the user controls and can be altered, so the back end must enforce the rules for security.
7	(1) Understand users and requirements; map the user flow and screens; wireframe; design the user interface; prototype and get feedback; build the front end; test and refine. (2) A wireframe is a simple layout without color or graphics; a prototype is a clickable model of the designed screens that users can try. (3) Fixing problems on a prototype is cheaper and faster than rewriting finished code.